Expression Tree

A few points:

In this chapter, we will examine how a postfix expression can be converted to an expression tree where the leaves are operands and internal nodes are operators. We will also examine how we can find the value of such an expression.

Algorithm:
Input : an expression in postfix. For simplicity, the operands are assumed to be single digits.
Process : construct an expression tree
Output: find the value of the expression

Phase 1:
Construct the expression tree:

Data structure used: stack of tree nodes

Scan the input left to right
If operand, create a tree node and push it to a stack of tree nodes.
If operator, pop two tree nodes from the stack, create a new tree node with the operator as the key.
Attach the popped nodes as subtrees of the new node.
Push the new node into the stack.

Phase 2 :
Evaluate the expression tree:
Stack should have one and only one tree node.
On this tree, do the evaluation.
If the node is a leaf, return the operand.
Else
    Evaluate the left subtree
    Evaluate the right subtree
    Apply the operator of this node on these two results

Details of the interface and the implementation:

Stack:

We are using an array implementation of the stack. Each element of the stack is a node which has two self referential members left_ and right_ standing for left and right subtrees.
It has a key field of char type.

This design has limitation that the operand can be one character. To avoid this, you may want to design a union of operand(int) and operator(char). You may not require a discriminator as the operands are in the leaves alone.

Stack structure:

```
#define MAXSIZE 100
struct node
{
        char ch_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;

struct stack
{
        node_t* key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```

stack interface:
We do observe a few changes compared to a normal stack.

Observe that pop returns the key of the stack which is a pointer to a node.

The function push_operand is similar to the push we have had earlier.

The function push_operator requires that two operands be popped, a new node created and these Two popped nodes be added as subtrees. So, we have made a separate function for pushing operators.

```
void init_stack(stack_t *ptr_stack);
void deinit_stack();
void push_operand(stack_t *ptr_stack, char ch);
void push_operator(stack_t *ptr_stack, char ch);
node_t* pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```

The eval function pops the one and only node from the stack.
Evaluates the left subtree recursively and the right subtree recursively and then combine the results based on the operator at the root.

```
int eval(stack_t *ptr_stack);
```

stack implementation:
create_node:
This helper function creates a node. If there is no memory, this function gives a message and exits the program.

```
static node_t* create_node(char ch)
{
	node_t *temp = (node_t*)malloc(sizeof(node_t));
	if(temp == NULL)
	{
		printf("memory full\n"); exit(1);
	}
	temp->ch_ = ch;
	temp->left_ = temp->right_ = NULL;
	return temp;
}
```

push_operand:
This function makes a node and pushes the node on to the stack.
The operand is stored as a single character.

```
void push_operand(stack_t *ptr_stack, char ch)
{
	if(! is_full(ptr_stack))
	{
		node_t* temp = create_node(ch);
		ptr_stack->key_[++ptr_stack->top_] = temp;
	}
	else
	{
		printf("stack full; cannot push\n");
	}
}
```

push_operator:

In this function, a new node is created with the operator as the key.
We pop two nodes from the stack and make them right and left subtrees of the new node created.
We then push the new node into the stack.

```
void push_operator(stack_t *ptr_stack, char ch)
{
	if(! is_full(ptr_stack))
	{
		node_t* temp = create_node(ch);
		temp->right_ = pop(ptr_stack);
		temp->left_ = pop(ptr_stack);
		ptr_stack->key_[++ptr_stack->top_] = temp;
	}
}
```

pop:
This function gives a message and returns NULL character if the stack is empty.
Otherwise returns the top element from the stack.
Observe this function returns a pointer to a node.

```
node_t* pop(stack_t *ptr_stack)
{
	if(! is_empty(ptr_stack))
	{
		return ptr_stack->key_[ptr_stack->top_--];
	}
	else
	{
		printf("stack empty; cannot pop\n");
		return '\0';
	}
}
```

Evaluation:

The eval function calls eval_recursive with the root of the tree – which is obtained by popping from the stack.
If the node is a leaf corresponding to the operand, the numeric value of the digit is returned.
This function in turn calls the evaluation of left and right subtrees recursively.
Once the results are obtained, based on the operator, the corresponding operation is performed.

```
int eval_recursive(node_t* root)
{

   int left, right;

   if (root->left_ == NULL && root->right_ == NULL)
      return root->ch_ - '0';

   left = eval_recursive(root->left_);
   right = eval_recursive(root->right_);

   switch (root->ch_)
   {
      case '+': return left + right;
      case '-': return left - right;
      case '*': return left * right;
      case '/': return left / right;
   }

   return 0;
}

int eval(stack_t *ptr_stack)
{
         return eval_recursive(pop(ptr_stack));
}
```

With this, the expression tree and its evaluation are complete.